Assignment 3: Testing, UX, and Development Plan

1 Project Identification

Team Name	Nazal
Team Members	Naman Anshumaan and Mohammad Fazlur Rahman
Roll Numbers	1020241465 and 1020242018
Assigned Project	Interactive Multi-Track Audio Editor and Mixer
Repository Link	`https://github.com/Ashoka-CS-3810-Design-Practices-in-CS/project-1-nazal.git`

1.1 Assigned Project

Problem Statement

Editing and arranging audio often requires professional digital audio workstations that can be difficult for beginners to understand. Our project aims to develop an easy-to-use multi-track audio editor where users can import audio files, arrange them on a timeline, edit individual clips, mix multiple tracks, apply basic effects, and export the final composition.

Motivation

The project combines interactive audio editing with concepts from data structures, algorithms, concurrency, data representation, and modular software architecture. Instead of treating audio processing as a sequence of commands, users can visually manipulate clips and tracks through a timeline-based interface.

Intended Users

The application is intended for students, beginners learning audio editing, users creating simple music compositions, and users working with voice recordings, podcasts, or multiple audio recordings.

Main Goal

The main goal is to create an intuitive, responsive, and modular application that allows users to import, arrange, edit, mix, and export a complete multi-track audio composition through a single interface.

2 Correctness and Testing Plan

We will use a combination of unit, integration, boundary, invalid-input, UI interaction, and performance testing to ensure that the system works correctly at both the component and application level. Unit tests will verify individual operations such as active clip detection, clip splitting, waveform calculations, and mixing, while integration tests will check complete workflows such as importing, editing, playing, saving, reopening, and exporting a project. Boundary and invalid-input testing will cover cases such as operations at clip boundaries, empty projects, unsupported or corrupted audio files, and missing audio sources. UI testing will focus on whether important user interactions, including creating tracks, manipulating clips, playback controls, volume, mute/solo, saving, and exporting, work as expected. Finally, performance testing will use increasingly larger projects to evaluate playback responsiveness, editing delays, waveform generation, export time, and overall UI responsiveness during playback.

	Test Input / Scenario	**Expected Output**	**Purpose**
T1	Import a valid WAV/MP3 file and place it on a track.	Audio clip is created at the selected timeline position and its waveform is displayed correctly.	Tests audio import, clip creation, and waveform generation.
T2	Split a clip at a valid position inside the clip.	The original clip is split into two representing the two portions of the original audio.	Tests the clip splitting algorithm and non-destructive editing.
T3	Place clips on multiple tracks with different start times and play the project.	All active clips play at their correct timeline positions and remain synchronized.	Tests active clip detection and multi-track playback.
T4	Change volume, mute one track, and solo another track during a multi-track composition.	The resulting audio reflects the correct volume, mute, and solo settings.	Tests the audio mixing algorithm and track controls.
T5	Save a project containing multiple tracks, clips, positions, volume settings, and fades. Close and reopen it.	The project is restored with the same structure, clip positions, and settings.	Tests project persistence and file handling.
T6	Attempt to import a corrupted or unsupported audio file.	The file is rejected and an appropriate error message is displayed without crashing the application.	Tests invalid-input handling.
T7	Trim or split a clip at its exact beginning, exact end, or outside its valid region.	Invalid operations are prevented and the project remains unchanged.	Tests boundary conditions and editing validation.

	Test Input / Scenario	Expected Output	Purpose
T8	Export a multi-track composition containing volume changes and fade effects.	A playable audio file is generated containing the correct arrangement, mixing, and effects.	Tests the complete playback/rendering and export pipeline.

3 Complexity and Performance Considerations

3.1 Active clip detection

Active clip detection determines which clips should currently be playing at a particular timeline position. The algorithm checks each clip and compares its start and end positions with the current playback time, giving a time complexity of $O(N)$, where N is the total number of clips. It requires $O(K)$ additional space to store the currently active clips, where K is the number of active clips. Since we expect the number of clips in a project to be relatively small, a linear scan is sufficient and keeps the implementation simple. The main potential bottleneck would occur in projects with a very large number of clips, where repeatedly scanning all clips could affect playback responsiveness. More advanced structures such as interval trees could improve search performance, but would add implementation complexity and memory overhead. We therefore accept the linear approach as a trade-off in favour of simplicity and reliability for the expected project size.

3.2 Waveform generation

Waveform generation processes audio samples and calculates the minimum and maximum amplitude in each display region. This takes $O(S)$ time, where S is the number of audio samples, and $O(P)$ space, where P is the number of waveform regions. Larger audio files will therefore require more processing time and memory, making waveform generation a potential bottleneck. Since this operation can take noticeable time for large files, it should not unnecessarily block normal UI interactions. Generated waveform information can also be retained or cached so that it does not need to be recalculated every time the waveform is displayed. This uses additional memory but improves responsiveness and avoids repeated computation. Here, we accept the extra memory usage as a trade-off for faster waveform display and a smoother user experience.

3.3 Clip editing

Moving, trimming, and changing fade parameters mainly involve modifying clip metadata and therefore take $O(1)$ time. Splitting a clip is also typically $O(1)$, although replacing or modifying elements in a vector can make the worst case $O(N)$. These operations require very little additional memory because the original audio is not modified and editing is non-destructive. Searching for or removing a clip can take $O(N)$. Since we expect a manageable number of clips per track, vectors provide a practical balance between performance and implementation simplicity. Using more complex data structures could improve searching or removal in larger projects, but would increase implementation complexity without providing much benefit at the expected scale. We therefore prioritise the simpler vector-based approach.

3.4 Multi-track audio mixing

Multi-track mixing applies volume, mute, solo, and fade settings before combining the audio samples from each track. For T tracks and S samples, this requires $O(T \times S)$ time and $O(S)$ space for the output buffer. As the number of tracks or audio length increases, the amount of processing also increases, making mixing one of the more important potential performance bottlenecks. The audio processing should therefore be designed separately from normal UI operations so that expensive mixing work does not make the interface unresponsive. We use the audio buffers and processing facilities provided by JUCE rather than implementing low-level audio handling ourselves. This gives up some low-level control in exchange for simpler implementation and more reliable audio processing, while the $O(T \times S)$ approach remains reasonable for the expected project size.

4 User Interface and UX Plan

The application will use a desktop DAW-style interface built using C++ and JUCE.

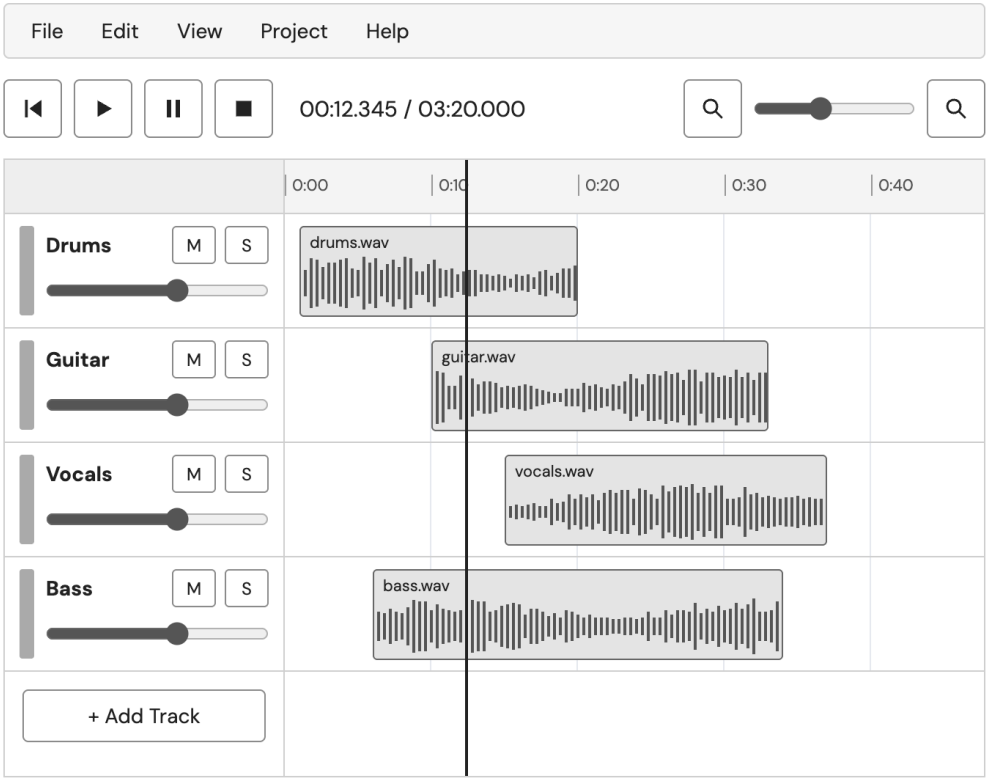


Figure 1: Project / Timeline view[1]

[1]This figure was generated using AI.

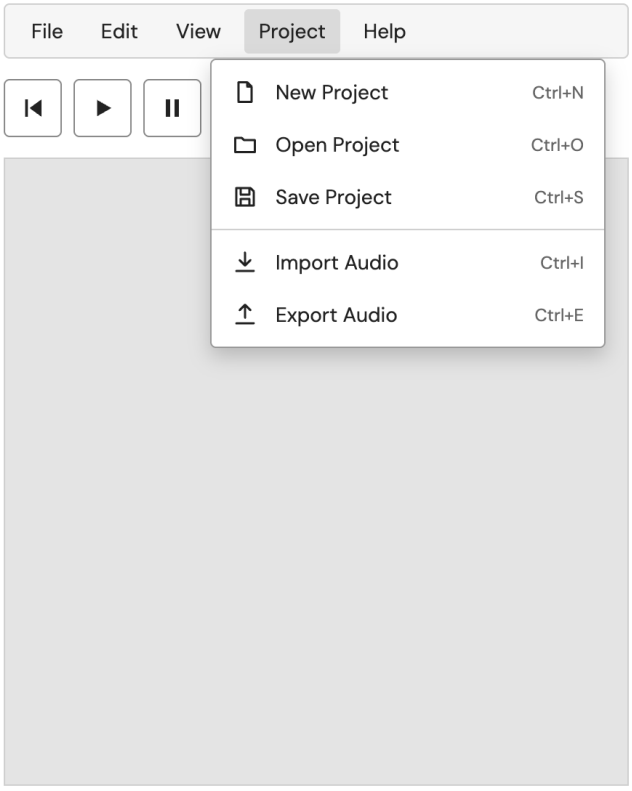


Figure 2: Project controls[2]

[2]This figure was generated using AI.

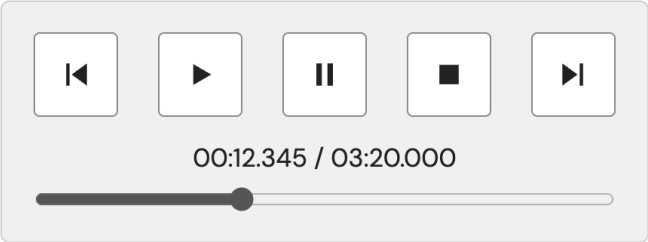


Figure 3: Transport controls[3]

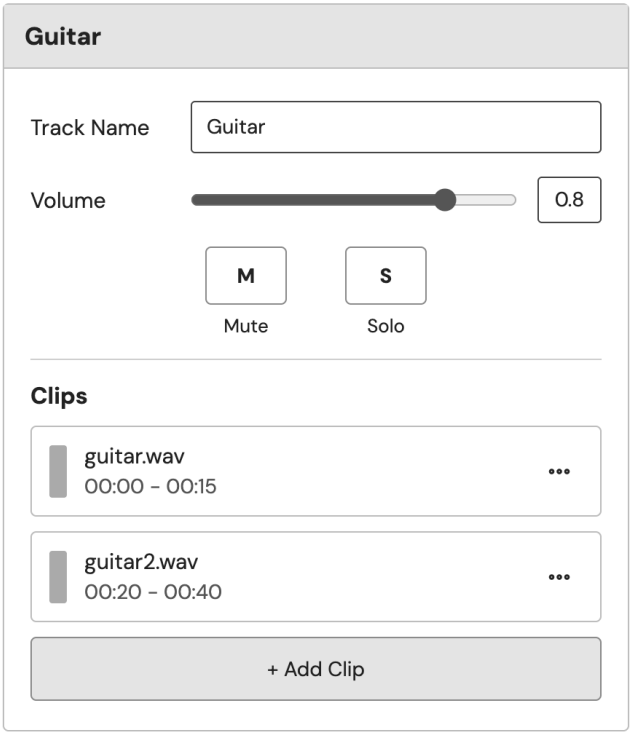


Figure 4: Track controls[4]

[3]This figure was generated using AI.
[4]This figure was generated using AI.

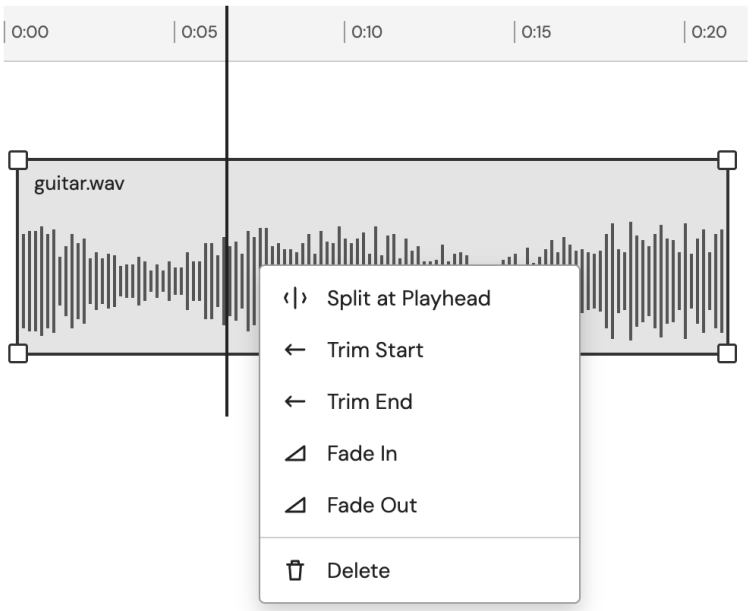


Figure 5: Clip editing[5]

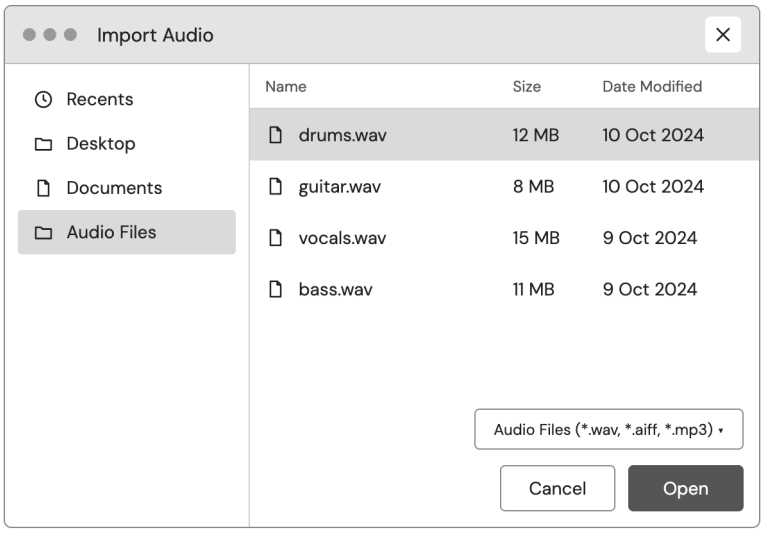


Figure 6: Audio import dialogue[6]

[5]This figure was generated using AI.
[6]This figure was generated using AI.

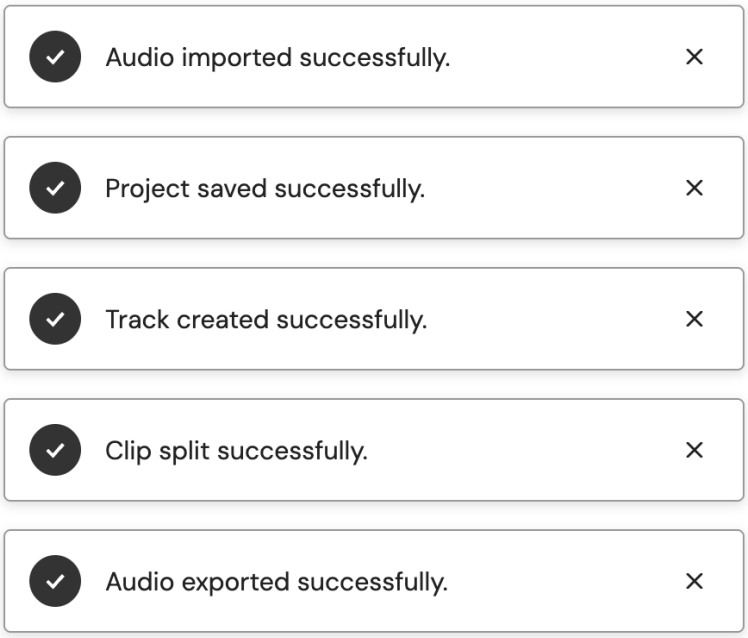


Figure 7: Feedback messages (Success)[7]

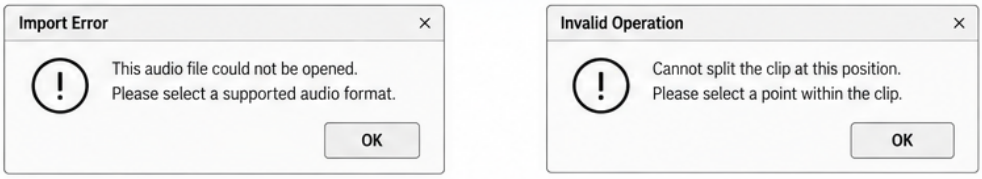


Figure 8: Error messages[8]

5 Risks, Assumptions, and Limitations

Technical risks

	Risk	**Impact**	**Mitigation**
R1	Real-time audio playback becomes unstable when multiple tracks play simultaneously.	Audio may stutter, lag, or become unsynchronized.	Keep the playback path lightweight, use JUCE's audio processing facilities, and test with increasing numbers of tracks.

[7]This figure was generated using AI.
[8]This figure was generated using AI.

	Risk	Impact	Mitigation
R2	Large audio files cause slow waveform generation or excessive memory use.	UI may become unresponsive and loading times may increase.	Generate/cache waveform information efficiently and avoid storing unnecessary duplicate audio data.
R3	Project files reference audio files that are later moved or deleted.	Reopening a project may result in missing clips.	Store clear audio-file references and detect missing files when loading a project, providing feedback to the user.
R4	Export produces incorrect audio compared to playback.	The final composition may not match what the user heard.	Reuse the same mixing, volume, mute/solo, and fade logic for both playback and export, and test exported files against known compositions.

Assumptions

1. Users will primarily work with supported common audio formats.
2. Users will have sufficient local storage and system resources for the audio files they import.
3. Projects are expected to contain a small-to-medium number of tracks and clips, rather than professional-scale sessions.
4. Users will keep referenced audio files accessible when reopening saved projects.
5. The application will primarily be used as a desktop application.

Anticipated limitations

1. The first version will support basic editing and effects rather than advanced professional audio processing.
2. The application may not perform efficiently with extremely large projects containing hundreds of tracks or very large numbers of clips.
3. The initial version will have a limited number of effects, primarily fade-in and fade-out.
4. Advanced features such as sophisticated automation, equalisation, compression, or real-time collaboration are outside the core scope.

6 Development Plan

We will develop the application incrementally so that the core data model and audio functionality are working before adding UI complexity.

Phase	Work	Responsible Member(s)	Expected Outcome
1	Review requirements and technical design	Naman	Finalised architecture, requirements, and implementation plan
2	Core data structures / system layer	Fazlur	Project, Track, Clip, audio representation, and basic JUCE setup
3	Algorithms and core functionality	Fazlur	Clip editing, active clip detection, waveform generation, mixing, and playback
4	UI/UX integration	Naman	Timeline, waveform display, track controls, transport controls, project/export interface
5	Testing and debugging	Naman + Fazlur	Unit, integration, boundary, invalid-input, and performance testing
6	Final refinement and documentation	Naman + Fazlur	Stable final application, documentation, demonstration, and assignments

We will also use the provided GitHub repository for collaborative development. We will make regular, meaningful commits rather than one large commit at the end. Work will be divided into logical features such as project/track/clip model, audio import, timeline, waveform, playback, editing, mixing, save/load, export, UI improvements, testing and bug fixes. Features will be tested before being merged into the main working version. This will also make it easier to identify which change introduced a bug.

7 Expected Deliverables

The final project deliverables will include the complete C++/JUCE source code, organised into modules, along with a working desktop audio editor application. We will provide documentation of the project, track, clip, and audio data representations to explain how the application stores and manages audio. The major algorithms will be documented, along with their time and space complexity, to explain the design choices and expected performance. We will also include testing documentation with representative test cases covering the main features and potential error conditions. Sample audio files and project files will be provided to demonstrate the application's functionality. Finally, the submission will include the documentation for Assignments 1, 2, and 3, covering the project requirements, design, implementation approach, testing strategy, and performance considerations.

8 Questions for Design Review

1. What is the single most important algorithmic challenge in your project?

The most important algorithmic challenge for our project has to be the multi-track audio mixing during playback and export. The system must identify the clips currently active on the timeline, apply track volume, mute/solo, and fade settings, and combine their audio samples into a single output.

2. What is the most appropriate data structure for the central problem, and why?

The most appropriate data structure is the clip, stored inside a track using a vector. A clip is central to our problem as it represents a region of an audio source through metadata such as its timeline position, duration, source offset, and fade settings. This allows editing operations to be non-destructive because the original audio does not need to be modified.

3. What is the most computationally expensive operation?

Multi-track audio mixing is the most computationally expensive as the operation samples from multiple active tracks must be processed and combined. Its straightforward implementation has a complexity of $O(T \times S)$.

4. What happens when the user provides invalid input?

To handle an invalid input, the application runs a validation check before performing the operation. Unsupported or corrupted audio files will be rejected, invalid clip operations will be prevented, and missing audio files will be detected when reopening projects. The user will receive clear feedback rather than allowing the application to crash.

5. Which component is most likely to change during future development?

In future development, we would update the UI layer to incorporate additional features such as visual improvements, keyboard shortcuts, waveform interactions, and optional features may be added as development progresses.

6. How does your architecture isolate that component?

The UI is in isolation from the core application logic. The UI sends user actions to components such as the project manager, timeline manager, editing manager, and playback controller rather than directly manipulating audio data. This allows the interface to change without requiring major changes to the underlying algorithms and data structures.

7. What is one design decision you considered but rejected? Why?

At first, we considered using React/Next.js for the build because it would make building the interface relatively straightforward and web accessible. However, we realised that the main challenge of this project is audio processing rather than web UI development. Using C++ with JUCE provides built-in support for audio devices, buffers, file handling, playback, audio formats, and GUI components, making it more suitable for the requirements of this project. We therefore chose C++ and JUCE to reduce the amount of low-level audio functionality we would need to implement ourselves.

8. What additional feature could be implemented without redesigning the core architecture?

An undo/redo system could be added without fundamentally changing the architecture. Since editing operations primarily modify clip and project metadata, the system could store previous project states and restore them.